Documento de Arquitectura de Software (DAS / SAD)

Plantilla oficial para proyectos individuales — arc42 simplificado + Modelo C4 + ADRs + Corte Vertical

Curso: Arquitectura de Software / Ingeniería de Software
Modalidad: Proyecto individual
Estudiante: _____
Proyecto: _____
Versión del documento: 1.0 — **Fecha:** ___ / ___ / _____

Instrucciones generales para el estudiante

Este documento es el **entregable arquitectónico oficial** de su proyecto individual. Debe diligenciarlo en su totalidad y acompañarlo de un **repositorio Git con código ejecutable** (Sección 5). No se evalúa la aplicación completa: se evalúa que la arquitectura **exista, esté justificada y se pueda ejecutar** mediante un **Corte Vertical (Vertical Slice / Walking Skeleton)**: una sola funcionalidad CRUD que atraviese todas las capas del sistema, desde la interfaz/API hasta la base de datos.

Reglas de oro:

1. Toda decisión técnica debe estar **justificada** con atributos de calidad (ISO/IEC 25010), nunca con "es más fácil" o "es la que conozco".
2. Todo diagrama debe ser **consistente** con el código entregado: si el diagrama dice PostgreSQL, el repositorio debe conectarse a PostgreSQL.
3. El documento se escribe en **lenguaje técnico de ingeniería**: preciso, medible y verificable.

SECCIÓN 1: INTRODUCCIÓN Y DRIVERS ARQUITECTÓNICOS

1.1 Visión general del negocio y alcance

Describa en máximo una página:

- **Problema de negocio:** ¿qué necesidad real resuelve el sistema? ¿Quién lo padece hoy y cómo lo resuelve sin software?
- **Propuesta de solución:** ¿qué hace el sistema en una frase? (formato sugerido: *"El sistema permite a [actor] realizar [capacidad] para lograr [beneficio medible]"*).
- **Alcance del proyecto individual (in-scope):** lista explícita de módulos/funcionalidades que SÍ se diseñan en esta arquitectura.

- **Fuera de alcance (out-of-scope):** lista explícita de lo que NO se aborda (ej. pasarela de pagos real, app móvil nativa, reportería avanzada). Declarar el fuera de alcance es una decisión arquitectónica, no una debilidad.

1.2 Historias de Usuario / Casos de Uso primarios (drivers funcionales)

Seleccione las **2 o 3 historias de usuario arquitectónicamente significativas**: aquellas que ejercitan las partes más críticas o riesgosas del sistema (no las más simples). Estas historias guiarán los diagramas de la Sección 3 y el Corte Vertical de la Sección 5.

ID	Historia de usuario (formato: Como… quiero… para…)	¿Por qué es arquitectónicamente significativa?
HU-01	Como *[rol]*, quiero *[acción]*, para *[beneficio]*	Ej.: ejercita autenticación, persistencia y validaciones de negocio
HU-02		
HU-03		

Criterio de selección: una historia es significativa si al implementarla se valida el patrón arquitectónico, la seguridad, la persistencia o la integración con un sistema externo. “Ver la pantalla de inicio” NO es significativa.

1.3 Atributos de Calidad prioritarios (ISO/IEC 25010) — especificación cuantitativa

Los atributos de calidad son los verdaderos **drivers de la arquitectura**. Seleccione **3 atributos prioritarios** de la ISO/IEC 25010 y especifíquelos como **escenarios de calidad medibles** (estímulo → entorno → respuesta → medida). Un atributo sin métrica no es evaluable y será devuelto.

#	Atributo (ISO 25010)	Escenario de calidad	Métrica objetivo (medible)	¿Cómo se verificará?
AC-01	Ej.: **Seguridad** (confidencialidad)	Un usuario no autenticado intenta acceder a `GET /api/clientes`	El sistema responde HTTP 401 en el 100 % de los intentos; las contraseñas se almacenan con hash	Prueba en Postman + inspección de la tabla `usuarios`

#	Atributo (ISO 25010)	Escenario de calidad	Métrica objetivo (medible)	¿Cómo se verificará?
		en operación normal	bcrypt (factor ≥ 10)	
AC-02	Ej.: **Eficiencia de desempeño** (tiempo de respuesta)	50 peticiones concurrentes al endpoint principal en servidor local	Percentil 95 del tiempo de respuesta ≤ 800 ms	Prueba de carga básica (ej. Apache JMeter / autocannon)
AC-03	Ej.: **Mantenibilidad** (modularidad)	Un desarrollador nuevo debe agregar un campo a la entidad principal	El cambio afecta máximo 3 archivos (modelo, migración, validación) y ninguna otra capa	Revisión de código del corte vertical

Catálogo de referencia ISO/IEC 25010: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, **seguridad**, **mantenibilidad**, portabilidad.

1.4 Restricciones

Las restricciones son decisiones que **ya vienen tomadas** y que la arquitectura debe acatar.

Tipo	Restricción	Origen / justificación
Técnica	Ej.: el motor de BD debe ser relacional y de licencia libre	Lineamiento del curso / costo
Técnica	Ej.: el backend debe exponer una API REST con JSON	Interoperabilidad con frontend
Operativa	Ej.: despliegue en máquina local o contenedor Docker; sin costos de nube	Recursos del estudiante
De tiempo	Ej.: corte vertical funcional en la semana N; documento final en la semana M	Calendario académico
Normativa	Ej.: tratamiento de datos personales conforme a Ley 1581 de 2012 (Colombia)	Habeas data

SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA

2.1 Patrón arquitectónico principal

Seleccione **un** patrón principal y justifíquelo contra sus atributos de calidad (Sección 1.3). Opciones típicas para un proyecto individual:

Patrón candidato	Fortalezas típicas	Costos típicos	¿Cuándo elegirlo?
MVC monolítico	Simplicidad, curva de aprendizaje baja, despliegue único	Acoplamiento si crece; escala vertical	CRUD académico con 1 desarrollador
N-Capas (presentación / negocio / datos)	Separación de responsabilidades, mantenibilidad	Más código de infraestructura	Cuando la lógica de negocio es no trivial
Arquitectura Limpia / Hexagonal	Independencia de framework y BD; testabilidad alta	Sobre-ingeniería para CRUDs simples	Cuando la mantenibilidad/testabilidad es AC prioritario
Microservicios	Escalabilidad y despliegue independiente	Complejidad operativa alta	**Casi nunca** en proyecto individual: debe justificarse de forma excepcional

Formato de justificación obligatorio (mínimo un párrafo):

> *"Se selecciona el patrón _____ porque el atributo de calidad prioritario AC-__ (_____) exige _____. Se descarta _____ porque su costo en _____ contradice la restricción _____."*

2.2 Pila tecnológica (Tech Stack) — tabla justificativa

Cada fila debe conectarse con un atributo de calidad o una restricción. "Porque lo vimos en clase" no es justificación arquitectónica (aunque puede ser una restricción de tiempo legítima: en ese caso, declárela como tal).

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC-xx / Restricción)
Lenguaje backend	Ej.: JavaScript (Node.js)	20 LTS	Java 21	Restricción de tiempo: dominio previo del estudiante; ecosistema npm acelera el corte vertical
Framewo	Ej.: Express	4.x	NestJS	AC-03 Mantenibilidad: estructura mínima suficiente para MVC; NestJS agrega complejidad no

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC-xx / Restricción)
rk backend				requerida
ORM / acceso a datos	Ej.: Sequelize	6.x	SQL nativo	AC-03: migraciones versionadas y modelo tipado; trade-off aceptado en rendimiento (ver ADR-001)
Base de datos	Ej.: PostgreSQL	16	MongoDB	AC-01 Integridad: el dominio exige relaciones y transacciones ACID
Frontend	Ej.: HTML + Bootstrap / React	—	—	Alcance: interfaz mínima para demostrar el flujo punta a punta
Autenticación	Ej.: JWT + bcrypt	—	Sesiones en servidor	AC-01 Seguridad + API sin estado (ver ADR batalla sesiones vs JWT)
Pruebas	Ej.: Postman + Jest	—	—	Verificación de AC-01 y AC-02
Control de versiones	Git + GitHub/GitLab	—	—	Restricción del curso

SECCIÓN 3: VISTAS DE ARQUITECTURA (MODELO C4 Y UML)

Toda vista debe incluir: **el diagrama**, una **leyenda** y un **párrafo de lectura** (qué debe entender quien lo mira en 30 segundos). Herramientas sugeridas: Structurizr, draw.io, PlantUML o Mermaid.

Nota: los diagramas de ejemplo de esta sección siguen la **notación estándar UML** y fueron elaborados con PlantUML. El estudiante puede elaborar los suyos en PlantUML, draw.io, StarUML o Visual Paradigm, siempre respetando la notación UML.

3.1 Vista de Contexto — C4 Nivel 1

Muestra el sistema como una caja negra: actores humanos, sistemas externos y las relaciones entre ellos. **No** muestra tecnología interna.

Ejemplo de referencia (adaptar al proyecto propio):

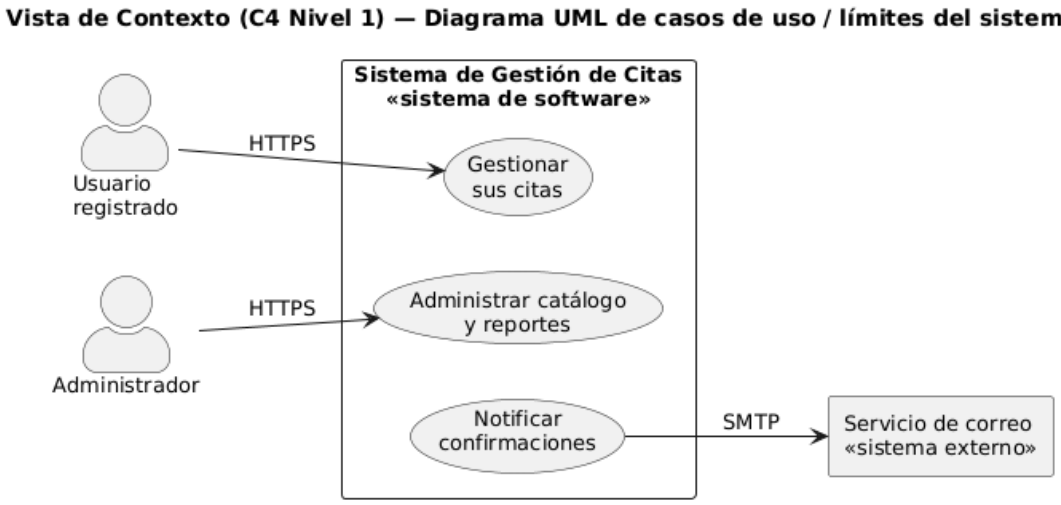


Diagrama UML de la Vista de Contexto (ejemplo de referencia)

Contenido obligatorio: todos los actores de las HU de la Sección 1.2; toda interfaz externa (correo, APIs de terceros); el protocolo de cada relación.

3.2 Vista de Contenedores / Capas — C4 Nivel 2

Abre la caja negra: aplicaciones ejecutables y almacenes de datos, con su tecnología concreta.

Ejemplo de referencia:

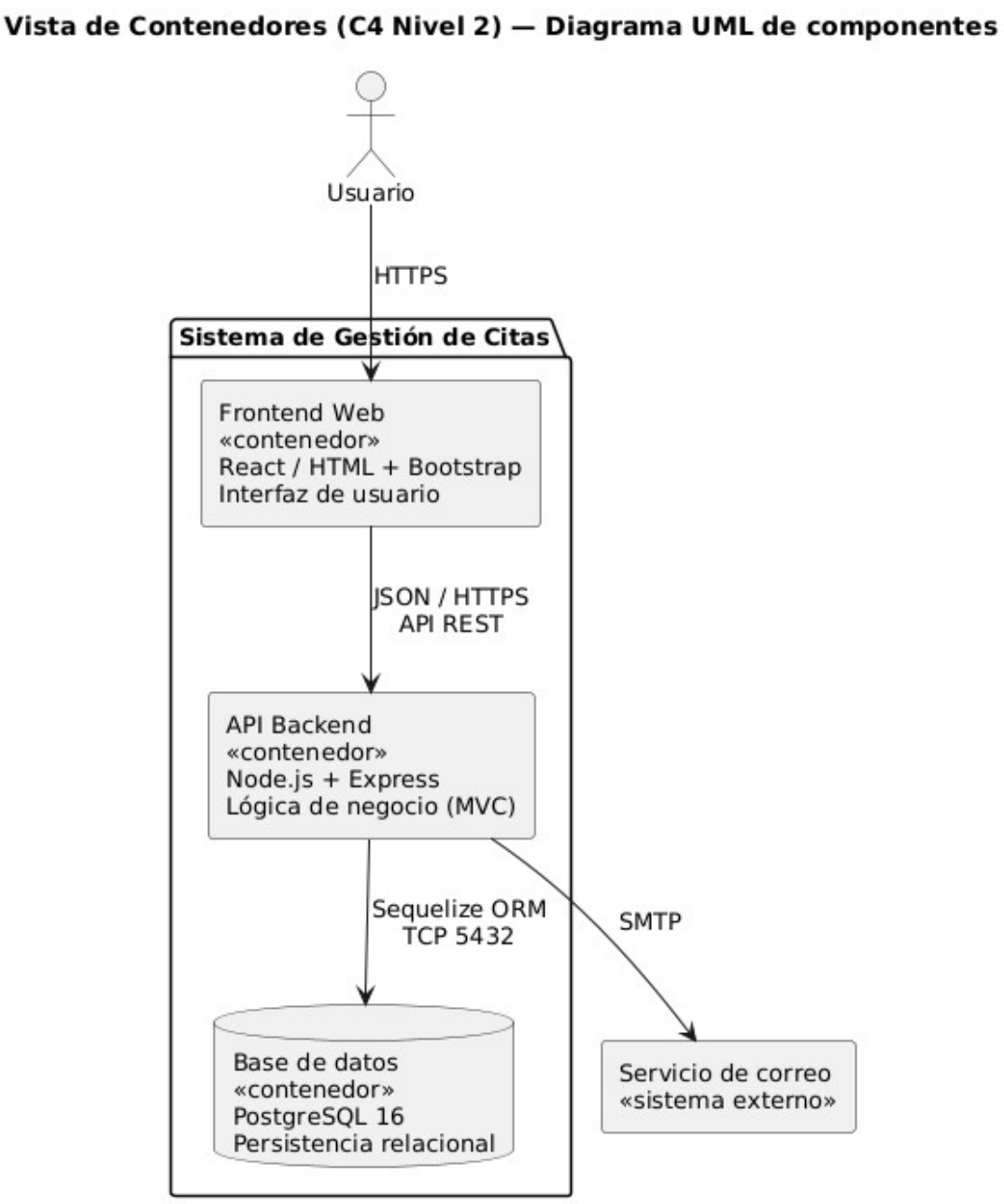


Diagrama UML de componentes — Vista de Contenedores (ejemplo de referencia)

Contenido obligatorio: cada contenedor con su tecnología entre corchetes; el protocolo/puerto de cada flecha; correspondencia exacta con el Tech Stack (Sección 2.2).

3.3 Vista de Dominio / Objetos — Diagrama de Clases UML

Modelo de datos que implementa el ORM. Mínimo: las entidades de las HU primarias, con atributos tipados, multiplicidades y llaves.

Ejemplo de referencia:

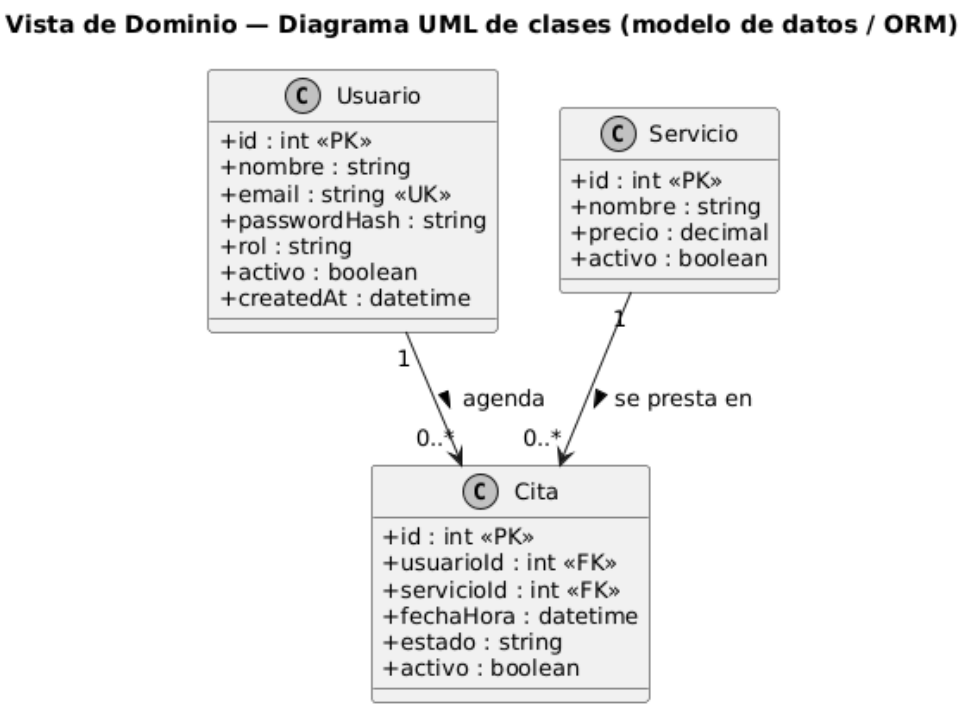


Diagrama UML de clases (ejemplo de referencia)

3.4 Vista Dinámica — Diagrama de Secuencia UML

Un diagrama de secuencia del **escenario primario principal** (la HU-01), mostrando cómo la petición atraviesa las capas del patrón elegido.

Ejemplo de referencia (crear cita — MVC):

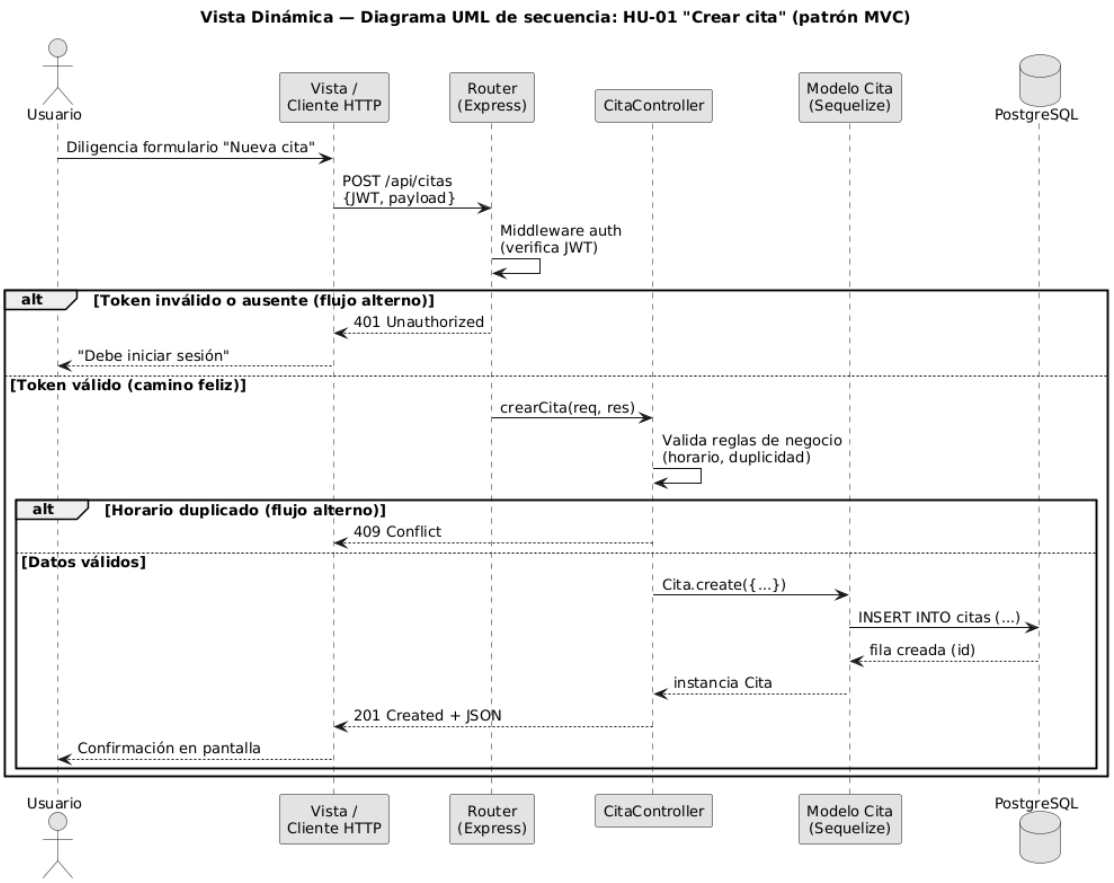


Diagrama UML de secuencia (ejemplo de referencia)

Contenido obligatorio: el camino feliz **y al menos un flujo alterno** (ej. 401 sin token, 409 por duplicidad).

3.5 Vista de Despliegue

Diagrama físico: nodos (máquinas/contenedores), qué artefacto corre en cada uno, puertos y red.

Ejemplo de referencia:

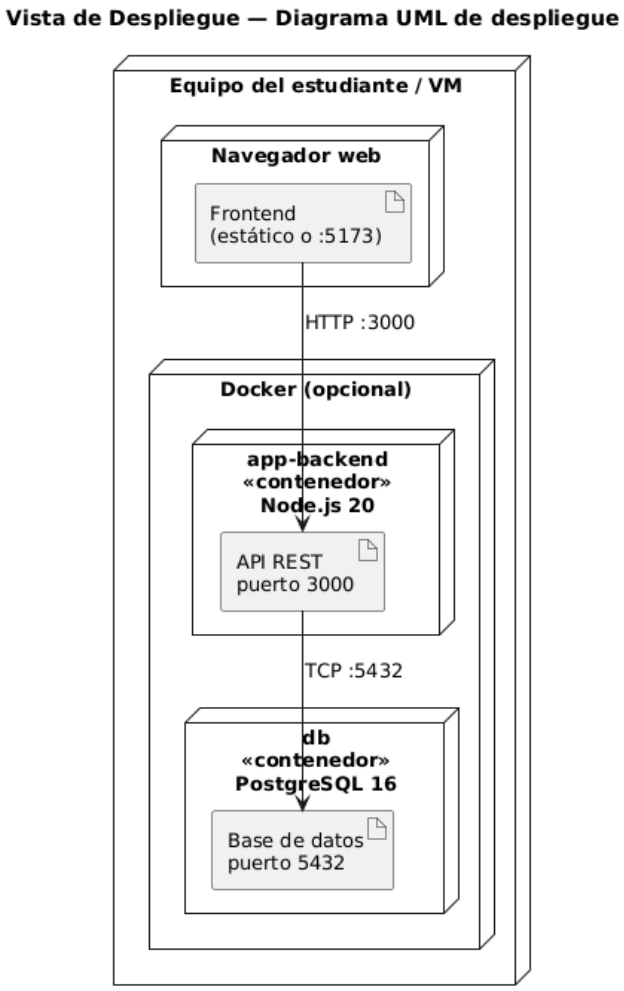


Diagrama UML de despliegue (ejemplo de referencia)

Contenido obligatorio: puertos expuestos, variables de entorno sensibles (solo el **nombre**, nunca el valor), y correspondencia con el `docker-compose.yml` o el manual de instalación del repositorio.

SECCIÓN 4: REGISTRO DE DECISIONES DE ARQUITECTURA (ADRs)

Documente **mínimo 1 decisión crítica** (se recomienda 2–3) usando la plantilla estándar en Markdown. Los ADRs viven en el repositorio en `docs/adr/` con nomenclatura `ADR-001-titulo-corto.md`, y son **inmutables**: si la decisión cambia, se crea un nuevo ADR que reemplaza (supersede) al anterior.

Plantilla obligatoria:

```
# ADR-001: [Título de la decisión, ej. "Uso de ORM Sequelize sobre SQL nativo"]
```

```
## Estatus
Aceptada | Propuesta | Rechazada | Reemplazada por ADR-XXX

## Fecha
AAAA-MM-DD

## Contexto
¿Qué problema o fuerza obliga a decidir? ¿Qué atributos de calidad
(AC-xx)
y restricciones están en juego? ¿Qué alternativas reales se
consideraron?

## Decisión
"Decidimos [opción elegida] porque [justificación ligada a AC-xx /
restricciones]."
Redactar en voz activa y en presente.

## Consecuencias / Trade-offs
- Positivas: ¿qué gana la arquitectura? (ej. +mantenibilidad,
+velocidad de desarrollo)
- Negativas: ¿qué se sacrifica y por qué es aceptable? (ej. -
rendimiento en consultas
  complejas; mitigación: consultas nativas puntuales vía raw queries)
- Deuda técnica asumida y condición para revisarla.
```

Decisiones candidatas típicas: patrón arquitectónico; SQL vs NoSQL; sesiones vs JWT; ORM vs SQL nativo; monolito vs servicios; borrado físico vs lógico.

SECCIÓN 5: ENTREGABLE DE PROGRAMACIÓN EJECUTABLE (CORTE VERTICAL)

El corte vertical demuestra que la arquitectura **funciona**, implementando **un (1) CRUD completo punta a punta** de la entidad principal de la HU-01: Interfaz/API → Controlador → Modelo/ORM → Base de datos → y de regreso.

5.1 Repositorio Git

Estructura mínima del repositorio:

```
proyecto-nombre/
├── docs/
│   ├── DAS.pdf                  # Este documento
│   └── adr/
│       └── ADR-001-*.md
├── db/
│   └── migrations/              # Scripts DDL versionados
```

```
├── src/
│   ├── config/                  # Conexión BD, variables de entorno
│   ├── models/                  # Capa Modelo (ORM)
│   ├── controllers/             # Capa Controlador
│   ├── routes/                  # Definición de endpoints
│   ├── middlewares/             # Auth, validación, manejo de errores
│   └── app.js                   # Punto de entrada
├── tests/
│   └── cita.test.js             # Pruebas de arquitectura
├── postman/
│   └── coleccion-corte-vertical.json
├── .env.example                 # Nombres de variables SIN valores
reales
├── .gitignore
├── package.json
└── README.md                    # Manual de instalación y ejecución
```

Política de commits (obligatoria):

- Commits **atómicos** y frecuentes (mínimo 10 commits significativos; un único commit "proyecto final" invalida la entrega).
- Formato *Conventional Commits*: `feat: agrega endpoint POST /api/citas`, `fix: corrige validación de fecha`, `docs: agrega ADR-001`, `test: prueba de integración del CRUD`.
- Rama `main` siempre ejecutable; trabajo en ramas `feature/*` con merge al finalizar.

`.gitignore` mínimo:

```
node_modules/
.env
*.log
dist/
coverage/
.DS_Store
```

> ⚠️ **Nunca** se versiona el archivo `.env` con credenciales. Su presencia en el historial de Git es causal de devolución del entregable.

5.2 Script SQL DDL / Migraciones

Creación física de las tablas con integridad referencial y **borrado lógico**. Ejemplo de referencia (PostgreSQL):

```
-- db/migrations/001_esquema_inicial.sql

CREATE TABLE usuarios (
    id              SERIAL          PRIMARY KEY,
    nombre          VARCHAR(120)    NOT NULL,
    email           VARCHAR(160)    NOT NULL UNIQUE,
    password_hash   VARCHAR(255)    NOT NULL,
    rol             VARCHAR(20)     NOT NULL DEFAULT 'cliente'
```

```
                    CHECK (rol IN ('cliente', 'admin')),
    activo          BOOLEAN         NOT NULL DEFAULT TRUE,   --
borrado lógico
    created_at      TIMESTAMP       NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMP       NOT NULL DEFAULT NOW()
);

CREATE TABLE servicios (
    id              SERIAL          PRIMARY KEY,
    nombre          VARCHAR(120)    NOT NULL,
    precio          NUMERIC(10,2)   NOT NULL CHECK (precio >= 0),
    activo          BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMP       NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMP       NOT NULL DEFAULT NOW()
);

CREATE TABLE citas (
    id              SERIAL          PRIMARY KEY,
    usuario_id      INTEGER         NOT NULL REFERENCES usuarios(id),
    servicio_id     INTEGER         NOT NULL REFERENCES servicios(id),
    fecha_hora      TIMESTAMP       NOT NULL,
    estado          VARCHAR(20)     NOT NULL DEFAULT 'pendiente'
                    CHECK (estado IN
('pendiente','confirmada','cancelada')),
    activo          BOOLEAN         NOT NULL DEFAULT TRUE,   --
borrado lógico
    created_at      TIMESTAMP       NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMP       NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_cita_horario UNIQUE (servicio_id, fecha_hora)
);

CREATE INDEX idx_citas_usuario ON citas(usuario_id);
```

Requisitos evaluables: PK en toda tabla; FKs con `REFERENCES`; al menos una restricción `CHECK` y una `UNIQUE`; campo `activo` de borrado lógico en toda entidad de negocio; índices justificados.

5.3 Código Backend MVC / ORM (ejemplo de referencia en Node.js + Express + Sequelize)

El estudiante adapta este esqueleto a su stack; lo evaluable es la **separación de capas**, no el lenguaje.

Modelo — `src/models/cita.model.js`:

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Cita = sequelize.define('Cita', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true
```

```
},
  usuarioId: { type: DataTypes.INTEGER, allowNull: false, field:
'usuario_id' },
  servicioId: { type: DataTypes.INTEGER, allowNull: false, field:
'servicio_id' },
  fechaHora: { type: DataTypes.DATE, allowNull: false, field:
'fecha_hora' },
  estado: {
    type: DataTypes.ENUM('pendiente', 'confirmada', 'cancelada'),
    defaultValue: 'pendiente'
  },
  activo: { type: DataTypes.BOOLEAN, defaultValue: true } // borrado
lógico
}, { tableName: 'citas', underscored: true });

module.exports = Cita;
```

Controlador — `src/controllers/cita.controller.js`:

```
const Cita = require('../models/cita.model');

// CREATE
exports.crear = async (req, res, next) => {
  try {
    const { servicioId, fechaHora } = req.body;
    if (!servicioId || !fechaHora) {
      return res.status(400).json({ error: 'servicioId y fechaHora son
obligatorios' });
    }
    const cita = await Cita.create({ usuarioId: req.usuario.id,
servicioId, fechaHora });
    return res.status(201).json(cita);
  } catch (err) {
    if (err.name === 'SequelizeUniqueConstraintError') {
      return res.status(409).json({ error: 'Ese horario ya está
ocupado' });
    }
    next(err);
  }
};

// READ (solo registros activos: borrado lógico)
exports.listar = async (req, res, next) => {
  try {
    const citas = await Cita.findAll({ where: { activo: true,
usuarioId: req.usuario.id } });
    return res.json(citas);
  } catch (err) { next(err); }
};
```

```
// UPDATE
exports.actualizar = async (req, res, next) => {
  try {
    const cita = await Cita.findOne({ where: { id: req.params.id,
activo: true } });
    if (!cita) return res.status(404).json({ error: 'Cita no
encontrada' });
    await cita.update(req.body);
    return res.json(cita);
  } catch (err) { next(err); }
};

// DELETE lógico
exports.eliminar = async (req, res, next) => {
  try {
    const cita = await Cita.findOne({ where: { id: req.params.id,
activo: true } });
    if (!cita) return res.status(404).json({ error: 'Cita no
encontrada' });
    await cita.update({ activo: false, estado: 'cancelada' });
    return res.status(204).send();
  } catch (err) { next(err); }
};
```

Rutas — `src/routes/cita.routes.js`:

```
const router = require('express').Router();
const ctrl = require('../controllers/cita.controller');
const auth = require('../middlewares/auth.middleware'); // verifica
JWT (AC-01)

router.post('/', auth, ctrl.crear);
router.get('/', auth, ctrl.listar);
router.put('/:id', auth, ctrl.actualizar);
router.delete('/:id', auth, ctrl.eliminar);

module.exports = router;
```

Punto de entrada — `src/app.js`:

```
require('dotenv').config();
const express = require('express');
const sequelize = require('./config/database');
const citaRoutes = require('./routes/cita.routes');

const app = express();
app.use(express.json());
app.use('/api/citas', citaRoutes);

// Manejo centralizado de errores (mantenibilidad AC-03)
app.use((err, req, res, next) => {
```

```
  console.error(`[ERROR] ${req.method} ${req.originalUrl}:`,
err.message);
  res.status(500).json({ error: 'Error interno del servidor' });
});

const PORT = process.env.PORT || 3000;
sequelize.authenticate()
  .then(() => app.listen(PORT, () => console.log(`API en
http://localhost:${PORT}`)))
  .catch((err) => console.error('No hay conexión a la BD:',
err.message));
```

5.4 Pruebas de la arquitectura

Se acepta **una** de las dos modalidades (o ambas):

a) Colección Postman (`postman/coleccion-corte-vertical.json`) con mínimo 6 peticiones, cada una con *tests* automáticos:

1. `POST /api/auth/login` → 200 y devuelve token
2. `POST /api/citas` **sin** token → **401** (valida AC-01 Seguridad)
3. `POST /api/citas` con token → **201**
4. `POST /api/citas` horario duplicado → **409** (valida regla de negocio)
5. `GET /api/citas` → 200 y la cita creada aparece
6. `DELETE /api/citas/:id` → 204 y la fila queda `activo = false` en la BD

b) Prueba de integración (`tests/cita.test.js`, ejemplo con Jest + Supertest):

```
const request = require('supertest');
const app = require('../src/app');

describe('Arquitectura del corte vertical: CRUD de citas', () => {
  test('rechaza peticiones sin JWT (AC-01 Seguridad)', async () => {
    const res = await request(app).post('/api/citas').send({});
    expect(res.status).toBe(401);
  });

  test('crea una cita atravesando todas las capas', async () => {
    const token = await obtenerTokenDePrueba();
    const res = await request(app)
      .post('/api/citas')
      .set('Authorization', `Bearer ${token}`)
      .send({ servicioId: 1, fechaHora: '2026-10-01T10:00:00Z' });
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id'); // la fila existe en la BD
  });
});
```

Criterio de aceptación del corte vertical: desde una terminal limpia, siguiendo únicamente el `README.md`, el docente debe poder instalar, migrar la BD, arrancar el servidor y ejecutar las pruebas **sin intervención del estudiante**.

SECCIÓN 6: EVALUACIÓN DE ARQUITECTURA Y TRADE-OFFS

6.1 Análisis de compromisos

Toda arquitectura sacrifica algo. Declare explícitamente el balance logrado:

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	¿Por qué es aceptable en este proyecto?
Ej.: Mantenibilidad (AC-03)	Rendimiento en consultas complejas	El ORM genera N+1 queries en el listado con relaciones	El volumen esperado (< 1.000 filas) mantiene el P95 bajo la meta de AC-02; se documenta mitigación en ADR-001
Ej.: Seguridad (AC-01)	Simplicidad / velocidad de desarrollo	Middleware JWT + hashing agregan ~3 archivos y complejidad al flujo	El sistema maneja datos personales (restricción normativa)

6.2 Puntos de sensibilidad y riesgos técnicos

Un **punto de sensibilidad** es un elemento de la arquitectura cuyo cambio impacta directamente un atributo de calidad. Identifique mínimo 3, con su riesgo asociado y mitigación:

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad / Impacto	Mitigación propuesta
R-01	Ej.: Conexión única a la BD sin pool configurado	Eficiencia de desempeño	Saturación de conexiones bajo concurrencia	Media / Alto	Configurar pool de Sequelize (max: 10) y probar con carga
R-02	Ej.: Secreto JWT en variable de entorno local	Seguridad	Fuga del secreto invalida toda la autenticación	Baja / Crítico	`.env` fuera de Git; rotación del secreto documentada
R-	Ej.: Restricción `UNIQUE`	Adecuación funcional	Un cambio del negocio	Media / Medio	La regla vive en la BD **y** en el

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad / Impacto	Mitigación propuesta
03	`(servicio_id, fecha_hora)`		(citas grupales) rompe el modelo		controlador; ADR que documenta la condición de revisión

6.3 Conclusión arquitectónica (máx. media página)

Cierre respondiendo: ¿la arquitectura entregada satisface los escenarios de calidad de la Sección 1.3 con evidencia del corte vertical? ¿Qué evolucionaría primero si el proyecto pasara a producción real?

Lista de chequeo de entrega

#	Entregable	¿Incluido?
1	DAS completo (Secciones 1–6) en PDF dentro de `docs/`	☐
2	3 escenarios de calidad ISO 25010 **con métricas**	☐
3	5 vistas: Contexto, Contenedores, Clases, Secuencia, Despliegue	☐
4	Mínimo 1 ADR en `docs/adr/` con la plantilla oficial	☐
5	Repositorio Git con ≥ 10 commits atómicos y `.gitignore` correcto	☐
6	Script SQL DDL con PK, FK, CHECK, UNIQUE y borrado lógico	☐
7	CRUD punta a punta ejecutable (Modelo–Controlador–Rutas–API)	☐
8	Colección Postman **o** pruebas de integración pasando	☐
9	`README.md` que permite instalar y ejecutar sin ayuda del autor	☐
10	Análisis de trade-offs y ≥ 3 puntos de sensibilidad	☐